Installing the Stripe app for Adobe Commerce

Overview

This guide walks you through installing the Stripe app for **Adobe Commerce** with Adobe App Builder and Adobe Commerce App Management. The app integrates Stripe payment processing with Adobe Commerce through out-of-process extensibility, so the payment app runs independently in Adobe I/O Runtime and integrates with your Adobe Commerce Storefront through GraphQL APIs, REST APIs, and Adobe I/O Events.

Supported platforms

The Stripe app supports both:

- **Adobe Commerce as a Cloud Service (SaaS)** - Fully managed cloud service
- **Adobe Commerce on cloud infrastructure and on-premises (PaaS)** - Self-hosted or managed infrastructure

> ***Note:*** *Throughout this documentation, "PaaS" is used as shorthand for "Adobe Commerce on cloud infrastructure and on-premises" to improve readability.*

> ***PaaS Users****: PaaS-specific sections are labeled* ***[PaaS Only]****. These include installing the* `module-out-of-process-payment-methods` *module and configuring integration permissions.*

Choose the appropriate setup instructions below based on your deployment type.

Supported payment flows

After installation, the app supports these merchant flows:

- Stripe payment methods enabled in the Stripe payment methods dashboard.
- Redirect-based payment methods when success and failure page URLs are configured.
- Automatic capture during checkout or manual capture by invoicing the order in Commerce Admin.
- Full and partial refunds by creating credit memos in Commerce Admin.
- Optional saved payment methods for supported payment methods.

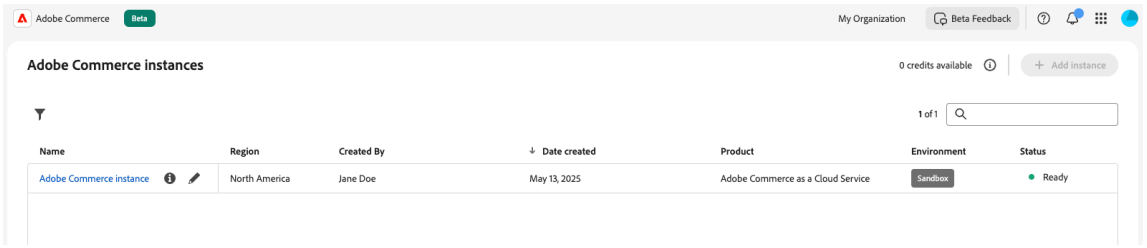
Before you begin

Set up Adobe Commerce Instance

[SaaS Only] Adobe Commerce as a Cloud Service

Before you begin, you will first need to have an instance of Adobe Commerce as a Cloud Service running. You can create one by:

1. Navigating to Experience Cloud Console
2. Under **Quick Access**, click **Commerce**
3. At the top right hand side of the page, click **Add instance**



If you are testing the app on a sandbox / development environment, you can import available sample data from Adobe to your Commerce instance.

[PaaS Only] Adobe Commerce on-premises/cloud infrastructure

For PaaS deployments, ensure you have:

1. Adobe Commerce 2.4.5+ installed and running. For installation guidance, see Adobe's Adobe Commerce installation guide.
2. Access to your Adobe Commerce Admin URL. For Adobe Commerce on cloud infrastructure, find the Admin URL in your Cloud project environment. See Adobe's Cloud Console guide. For on-premises environments, use the Admin URL configured for your Commerce installation.
3. API credentials for the authentication method your PaaS environment uses:
   - IMS authentication: use the OAuth Server-to-Server credential from Adobe Developer Console.
   - Commerce integration authentication: create or open the integration in **System** > **Extensions** > **Integrations** in Commerce Admin, activate it, and copy the Consumer Key, Consumer Secret, Access Token, and Access Token Secret. See Adobe's Commerce integration guide.
4. Required out-of-process payment, eventing, IMS, and Admin UI SDK modules installed from the Commerce application root directory:

```
# Run from the Adobe Commerce application root directory, where bin/magento
exists
composer require magento/module-out-of-process-payment-methods:^0.4.0
composer require magento/commerce-eventing
composer require magento/commerce-backend-sdk:^3.0
composer require magento/adobe-ims

php bin/magento module:enable \
  Magento_OutOfProcessPaymentMethods \
  Magento_AdobeCommerceEventsClient \
  Magento_AdobeCommerceEventsGenerator \
  Magento_AdobeCommerceEventsMeta \
  Magento_AdminUISDK \
  Magento_AdobeIms \
  Magento_AdminAdobeIms

php bin/magento setup:upgrade
```

```
php bin/magento setup:di:compile
php bin/magento cache:flush
```

Required PaaS module versions:

Module	Required version
`magento/module-out-of-process-payment-methods`	`^0.4.0`
`magento/commerce-eventing`	Version compatible with Adobe Commerce 2.4.5+
`magento/commerce-backend-sdk`	`^3.0`
`magento/adobe-ims`	Version compatible with Adobe Commerce 2.4.5+

5. Admin UI SDK v3.0+ configured for IMS authentication compatibility. For PaaS IMS setup, enable Adobe IMS from the Commerce application root directory:

```
# Run from the Adobe Commerce application root directory, where bin/magento exists
php bin/magento admin:adobe-ims:enable
php bin/magento cache:flush
```

When prompted, enter the Organization ID, Client ID, and Client Secret from the OAuth Web App credential in Adobe Developer Console. Then verify **Stores** > **Configuration** > **Adobe Services** > **Admin UI SDK** shows Admin UI SDK enabled and the **Configure extensions** button is available.

6. Verify the required modules are installed and enabled:

```
# Run from the Adobe Commerce application root directory, where bin/magento exists
composer show magento/module-out-of-process-payment-methods
composer show magento/commerce-eventing
composer show magento/commerce-backend-sdk
composer show magento/adobe-ims

php bin/magento module:status Magento_OutOfProcessPaymentMethods
php bin/magento module:status Magento_AdobeCommerceEventsClient
php bin/magento module:status Magento_AdobeCommerceEventsGenerator
php bin/magento module:status Magento_AdobeCommerceEventsMeta
php bin/magento module:status Magento_AdminUISDK
php bin/magento module:status Magento_AdobeIms
php bin/magento module:status Magento_AdminAdobeIms
```

> ***Important**: The `magento/module-out-of-process-payment-methods` module is required for PaaS deployments. Without it, Stripe configuration saves will fail with 404 errors.*

For sample data on PaaS environments, refer to the official Adobe Commerce sample data guide.

Set up your Stripe account

You can register for a Stripe account via Stripe's website. After registering, you can access your Stripe API keys via:

- Test Mode: https://dashboard.stripe.com/test/apikeys
- Live Mode: https://dashboard.stripe.com/apikeys

Configure Authentication

IMS Authentication (Recommended)

IMS (Identity Management Service) authentication is the recommended approach for both SaaS and PaaS deployments:

- **[SaaS Only]** IMS authentication is required. However, the current implementation allows Commerce integration authentication as a fallback if `OAUTH_CLIENT_ID` is missing. This fallback behavior may change in future releases.
- **[PaaS Only]** IMS authentication enables easier migration to SaaS and provides access to Admin UI SDK features

For both deployment types, IMS authentication allows the app to:

- Access Adobe Commerce APIs securely
- Integrate with Adobe Admin UI SDK v3.0+
- Work within Experience Cloud Shell environments

[PaaS Only] Integration Authentication (Alternative)

PaaS deployments can alternatively use Commerce Integration authentication (OAuth 1.0a):

1. Navigate to **System** > **Extensions** > **Integrations** in your Commerce Admin
2. Click **Add New Integration**
3. Fill in the integration details.
4. On the **API** tab, grant the permissions required for out-of-process payment methods, including `Magento_OutOfProcessPaymentMethods`.
5. Save and activate the integration.
6. Copy the Consumer Key, Consumer Secret, Access Token, and Access Token Secret from the activated integration. These values map to `COMMERCE_CONSUMER_KEY`, `COMMERCE_CONSUMER_SECRET`, `COMMERCE_ACCESS_TOKEN`, and `COMMERCE_ACCESS_TOKEN_SECRET` in `.env`.

Note: While integration authentication works for PaaS, Adobe recommends using IMS authentication to ease future migration to SaaS.

Set up App Builder

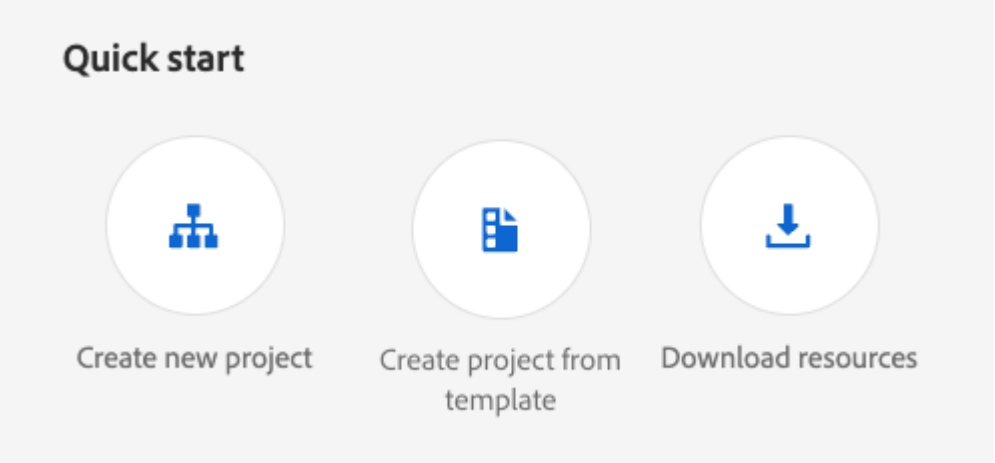
Obtain a license

You will need a license for App Builder in order to deploy the app. App Builder is necessary so that you can configure IMS authentication credentials, configure Commerce I/O Events, and persist data to Adobe I/O Lib State.

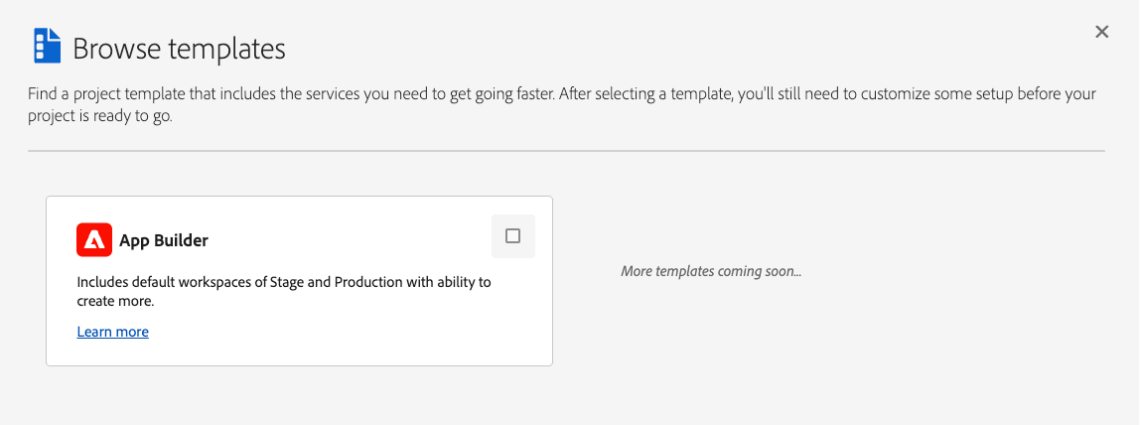
You can sign up to a trial of App Builder via https://developer.adobe.com/app-builder/trial/.

Create the project

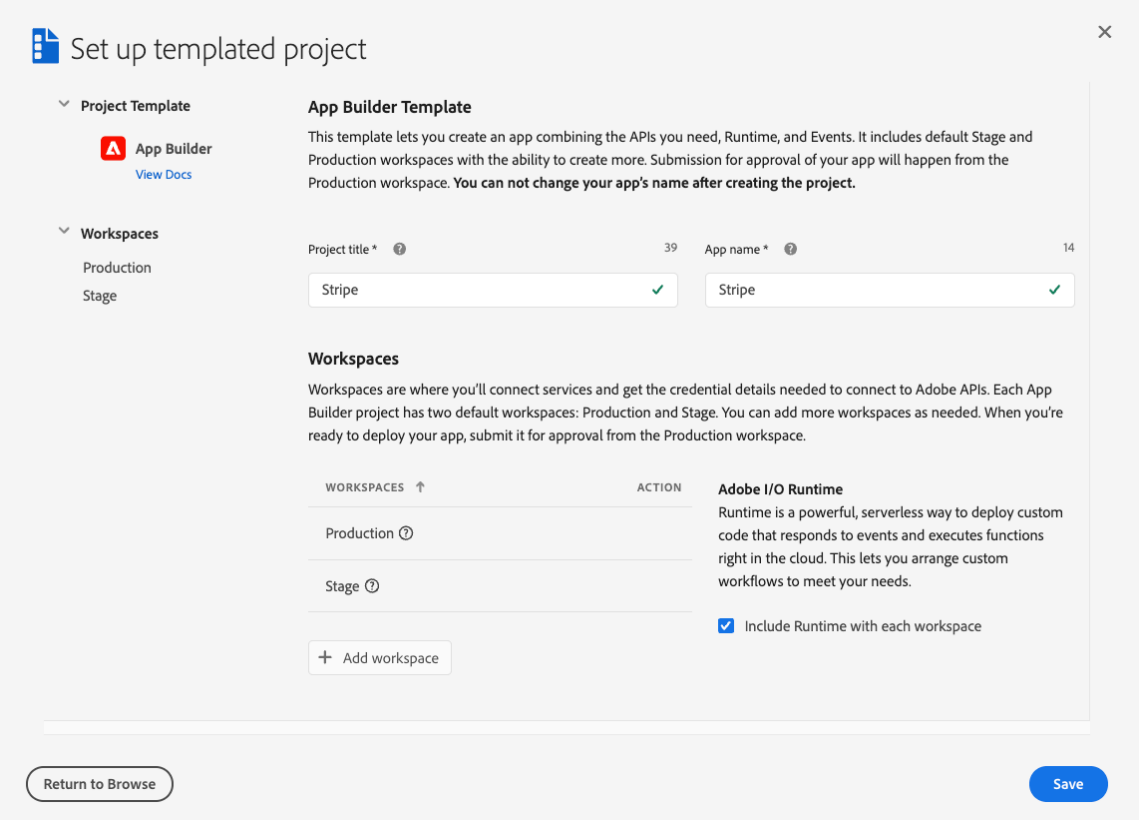
Once you have an App Builder license, navigate to [Adobe Developer Console](). You should see there a **Create project from template** button, indicating that your App Builder license is active:



Click on the **Create project from template** button, and select App Builder as a template:

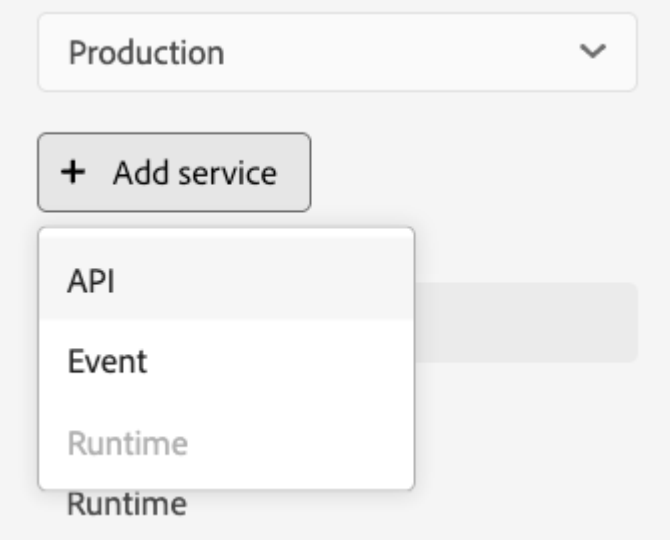


Fill the project details and click save:

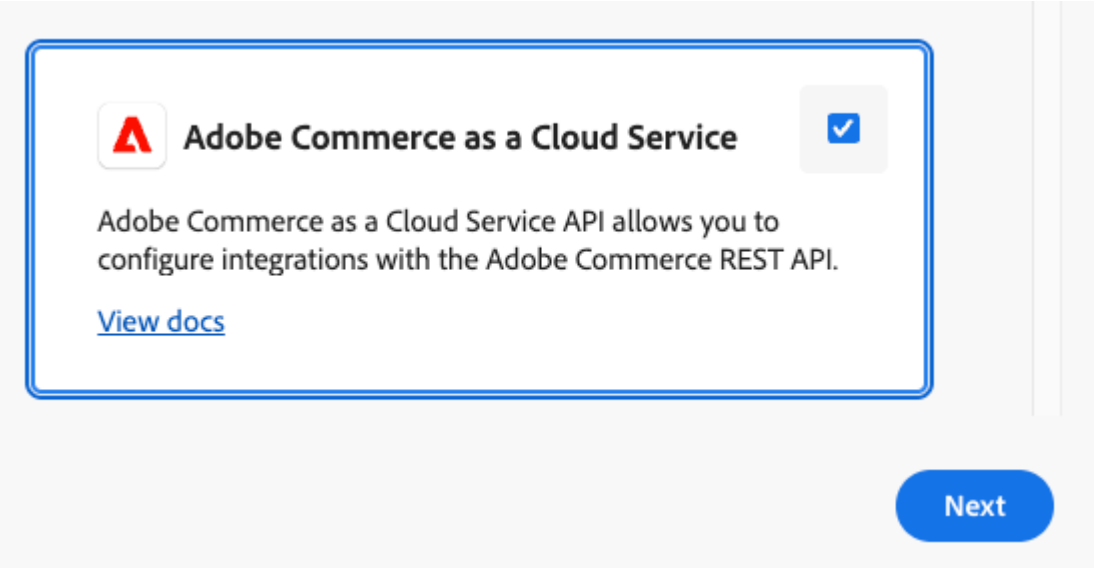


Add required APIs to the project

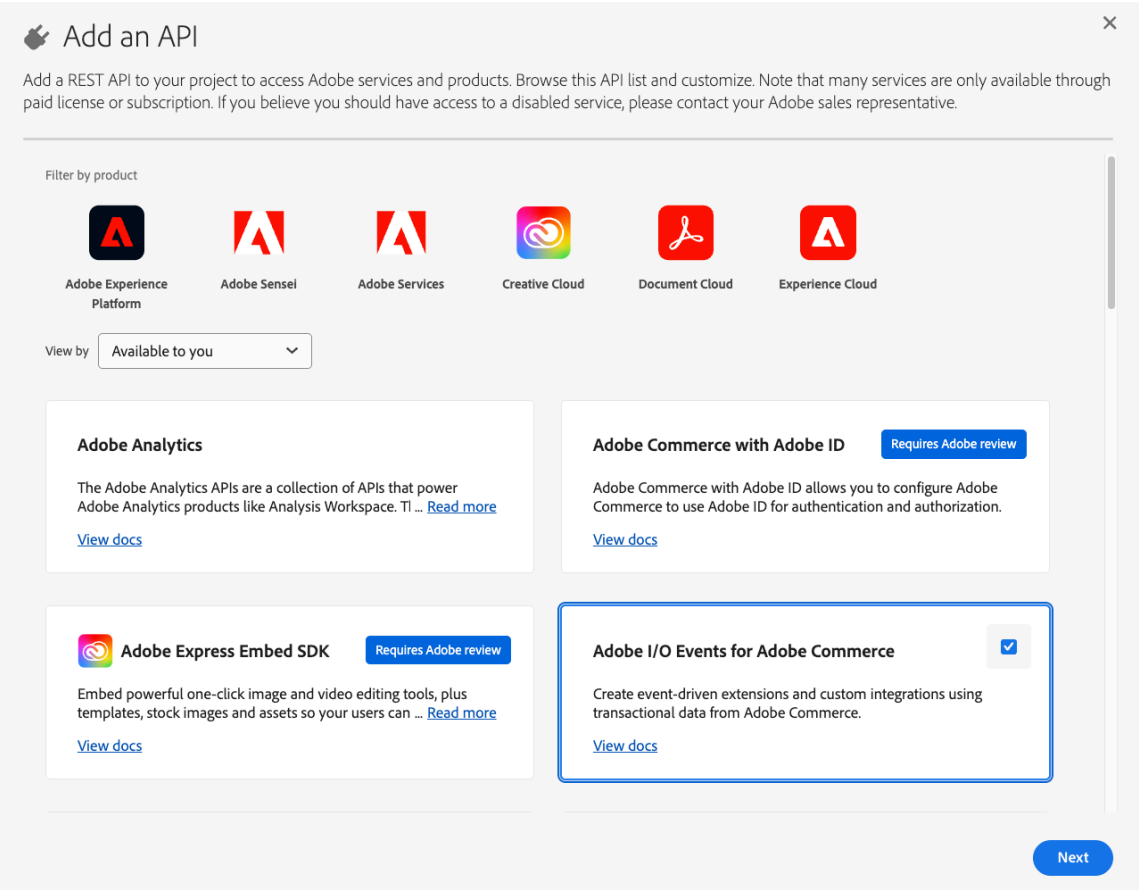
1. Select an environment, such as **Stage** or **Production**, click the **Add service** button, and then select **API**.



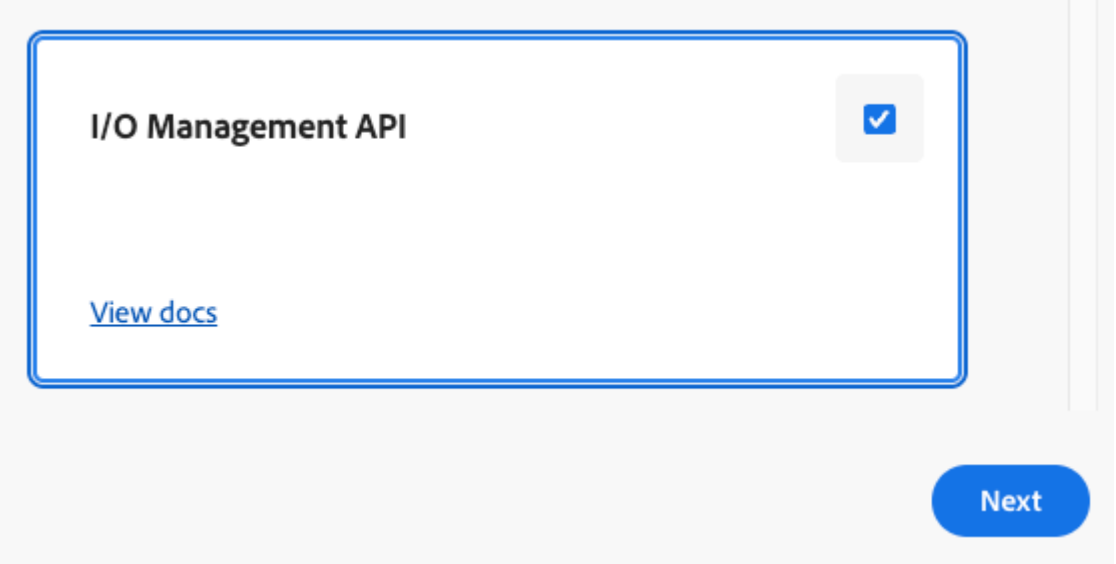
1. Add **Adobe Commerce as a Cloud Service** to the environment by selecting **Add service** > **API** > **Adobe Commerce as a Cloud Service**. This API provides the Commerce scopes used by the app's OAuth credentials.



1. Add **Adobe I/O Events for Adobe Commerce** to the environment by selecting **Add service** > **API** > **Adobe I/O Events for Adobe Commerce**. This API supports Commerce event provider and registration setup.



1. Add **I/O Management API** to the environment by selecting **Add service** > **API** > **I/O Management API**. The setup scripts and runtime actions use this API to read workspace metadata and manage event configuration.



1. Add **Adobe Commerce with Adobe ID** to the environment by selecting **Add service** > **API** > **Adobe Commerce with Adobe ID**. This API is used together with the **OAuth Web App** credential that you add in Add workspace credentials.

Do not create Commerce event registrations manually for a standard App Management installation. App Management creates the Commerce event provider and event registrations automatically when the app is installed.

Add workspace credentials

After creating the workspace and adding the required APIs, add the credentials required by the app.

1. In Adobe Developer Console, open the project and workspace where the app will be deployed.
2. Click **Add service** and select **OAuth Server-to-Server**.
3. Configure the OAuth Server-to-Server credential for the Adobe APIs used by the app, including Adobe Commerce as a Cloud Service, Adobe I/O Events for Adobe Commerce, and I/O Management API.

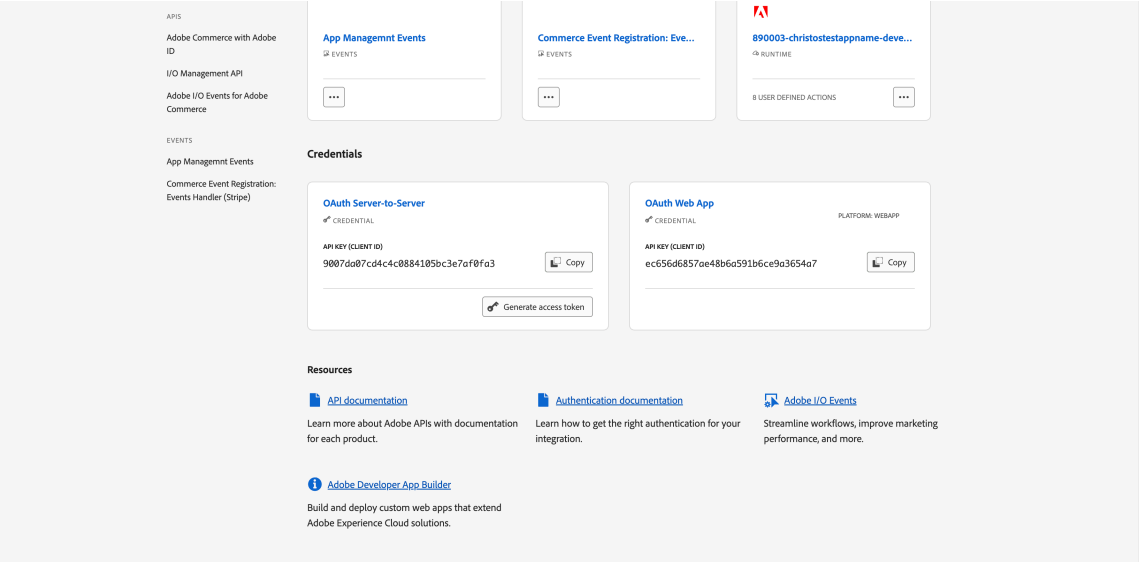


1. Save the credential.
2. Confirm the workspace now shows an **OAuth Server-to-Server** credential.

   The app runtime actions use this credential to call Adobe APIs. The `npm run sync-oauth-credentials` command must be run from the Stripe app project root directory, where `app.config.yaml` and `.env` are located. It reads this credential from the selected

workspace and writes the matching `OAUTH_*` values to `.env`. For more information, see Adobe's [OAuth Server-to-Server authentication documentation].

3. Click **Add service** again and select **OAuth Web App**.
4. Configure the OAuth Web App credential for the same workspace.



1. Save the credential.
2. Confirm the workspace now shows an **OAuth Web App** credential.

   Adobe Commerce App Management and the Commerce Admin association flow use this credential.

Use the credentials from your own Adobe Developer Console workspace. Do not reuse API keys or client IDs from another organization, project, or workspace.

Set up your local environment

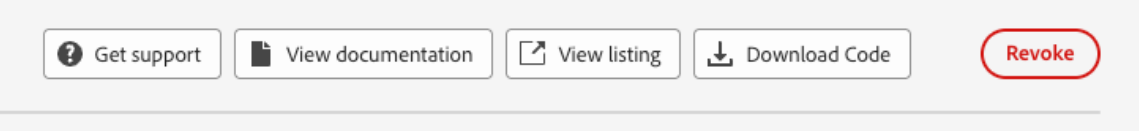
Before you can deploy the app to App Builder, you will additionally need to install **Node.js** and the **Adobe I/O CLI** package. The app supports Node.js `22.23.4` or later within the 22.x line, and Node.js `24.22.1` or later within the 24.x line (`>=22.23.4 <23.0.0 || >=24.22.1 <25.0.0` , as declared in the `engines` field of `package.json`). Please follow [the official instructions] to set up your local environment.

Install the App via Adobe Exchange Marketplace and App Management

To install the Stripe app, follow the steps below:

1. Navigate to the app's listing on the [Adobe Exchange Marketplace]
2. Click on the app listing to view it.
3. At the top right hand side, click the **Free** button and follow the instructions to install the app into your organization's environment.

4. The app will appear under Adobe Exchange App Management. If you are an account admin, click the **Review** button on the app page in order to approve it.
5. At the app's page, you will see a **Download Code** button at the top bar. Click it to download the app's code locally. You will need the code to set up your OAuth Credentials.



1. Extract the downloaded code to a local directory, and use the CLI to select the same Adobe Developer Console organization, project, and workspace:

   ```
   # Navigate to the extracted project directory
   cd path/to/extracted/stripe-app

   # Login and configure Adobe CLI (run from the project root directory)
   aio login -f
   aio console org select
   aio console project select
   aio console workspace select
   aio app use --merge
   aio where
   ```

   Confirm that `aio where` shows the workspace where you want to deploy the app.

2. Confirm that the `.env` and `.aio` files were added to the project directory, and run the following command:

   ```
   # Run from the Stripe app project root directory
   npm run sync-oauth-credentials
   ```

   This reads the selected workspace's **OAuth Server-to-Server** credential and generates these entries in the `.env` file:

   ```
   OAUTH_CLIENT_ID=
   OAUTH_CLIENT_SECRETS=[""]
   OAUTH_TECHNICAL_ACCOUNT_ID=
   OAUTH_TECHNICAL_ACCOUNT_EMAIL=
   OAUTH_SCOPES=[""]
   OAUTH_IMS_ORG_ID=
   ```

   These values come from the OAuth Server-to-Server credential in Adobe Developer Console. The deployed app's runtime actions read them to create access tokens for the Adobe Commerce Admin API. For more information, see Adobe's OAuth Server-to-Server authentication documentation.

3. Create the App Management configuration encryption key. App Management uses it to encrypt the secret key and webhook signing secret that you save in Adobe Commerce Admin, and the

Stripe runtime actions use it to read them. The key is required: without a valid key, saving or reading these secrets fails.

1. Generate the key:

   ```
   # Run from the Stripe app project root directory
   npx aio-commerce-lib-config encryption setup
   ```

   The command creates `.env` if it does not exist and adds a randomly generated 256-bit key (64 hexadecimal characters) to it:

   ```
   AIO_COMMERCE_CONFIG_ENCRYPTION_KEY=
   ```

   If `.env` already contains a key, the command keeps it and does not generate a new one.

2. Confirm that the key is present and valid:

   ```
   # Run from the Stripe app project root directory
   npx aio-commerce-lib-config encryption validate
   ```

3. Copy the key from `.env` to a secure place, such as your team's secret manager. Secrets saved in App Management cannot be decrypted without it.

Follow these rules for the key:

- Use one key for each App Builder workspace. If you deploy the same workspace from more than one `.env` file, for example one for SaaS and one for PaaS, every file must contain the same value.
- Do not change or remove the key after you have saved the Stripe configuration. A different key cannot decrypt the saved secrets. See App Management Configuration Fails to Load After an Encryption Key Change.
- Do not commit `.env` to source control.
- The deployment passes the key to the app's runtime actions. After you add or change it, deploy again.

4. **Configure deployment type and Commerce instance settings** in your `.env` file:

   Required Configuration:

   ```
   # Set your deployment type
   COMMERCE_DEPLOYMENT_TYPE=saas  # or "paas" for PaaS deployments

   # SaaS: https://<region>.api.commerce.adobe.com/<tenant-id>/
   # PaaS: https://your-commerce-domain.com/rest
   COMMERCE_BASE_URL=

   # Exact HTTPS storefront origins permitted to call checkout actions.
   # Separate multiple origins with commas.
   ALLOWED_ORIGINS=https://your-storefront.example
   ```

[PaaS Only] Additional Configuration for Integration Authentication:

If using Integration authentication instead of IMS, add these credentials:

```
# Commerce Integration credentials (PaaS only - alternative to IMS)
COMMERCE_CONSUMER_KEY=your_consumer_key
COMMERCE_CONSUMER_SECRET=your_consumer_secret
COMMERCE_ACCESS_TOKEN=your_access_token
COMMERCE_ACCESS_TOKEN_SECRET=your_access_token_secret
```

Note: For PaaS deployments, you can use either IMS authentication (recommended) or Integration authentication. If both are configured, the app will prefer IMS authentication.

5. Install app dependencies:

```
# Run from the Stripe app project root directory
npm install
```

6. Deploy the updated app:

```
# Run from the Stripe app project root directory
aio app deploy --force-build --force-deploy
```

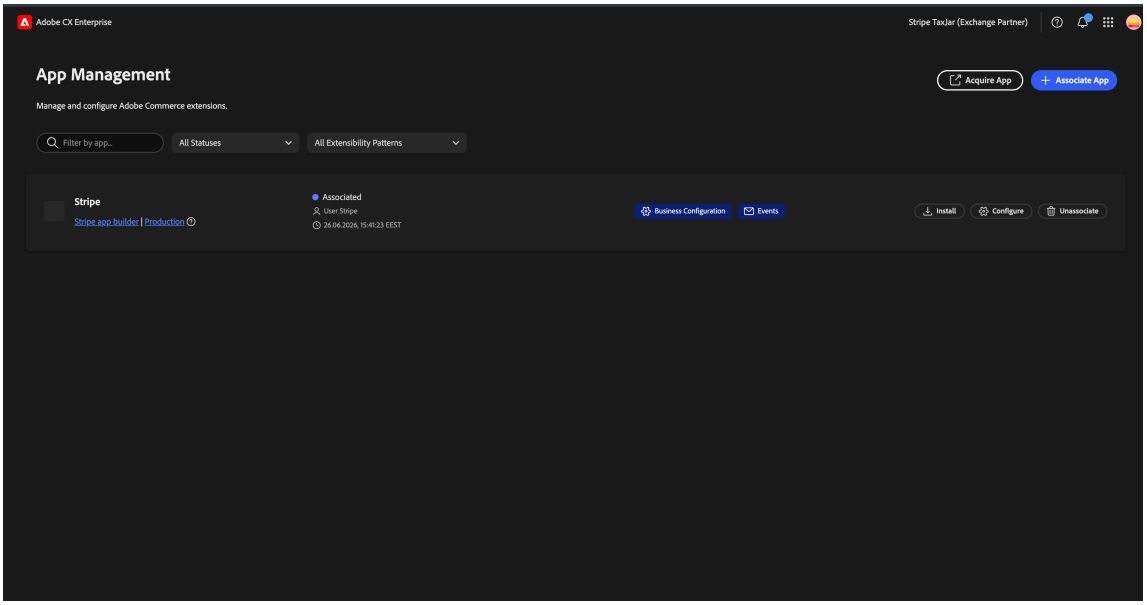
Do not use `--force-events` unless you intentionally want Adobe I/O Events providers and registrations recreated.

The deployment passes `AIO_COMMERCE_CONFIG_ENCRYPTION_KEY` from `.env` to the app's runtime actions. Deploy again whenever you add or change that value.

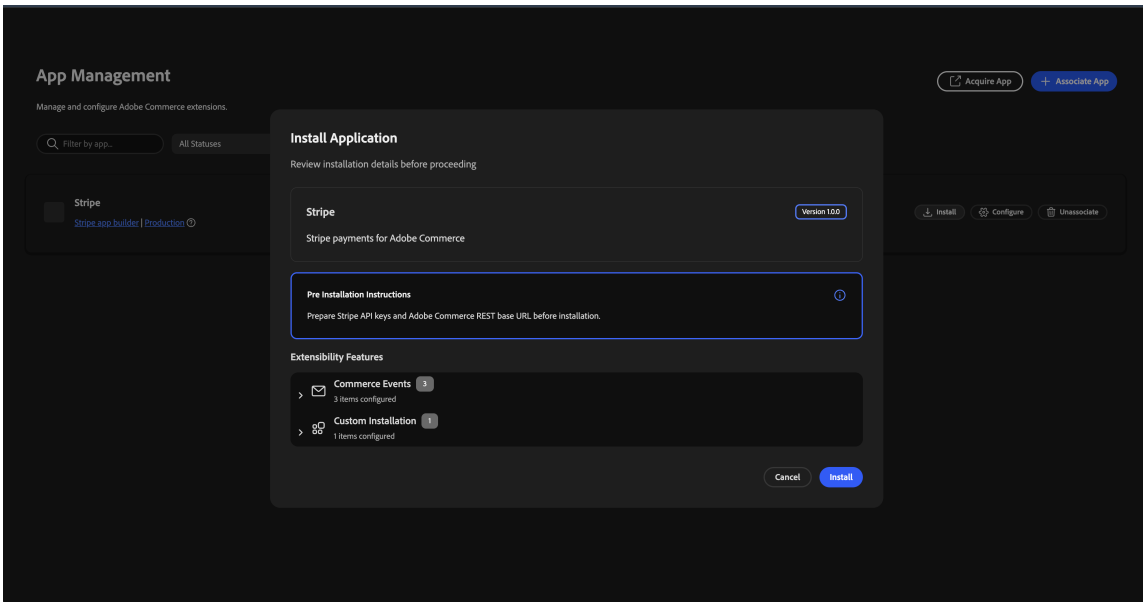
Install the App in Adobe Commerce App Management

After deployment, associate and install the app in Adobe Commerce Admin:

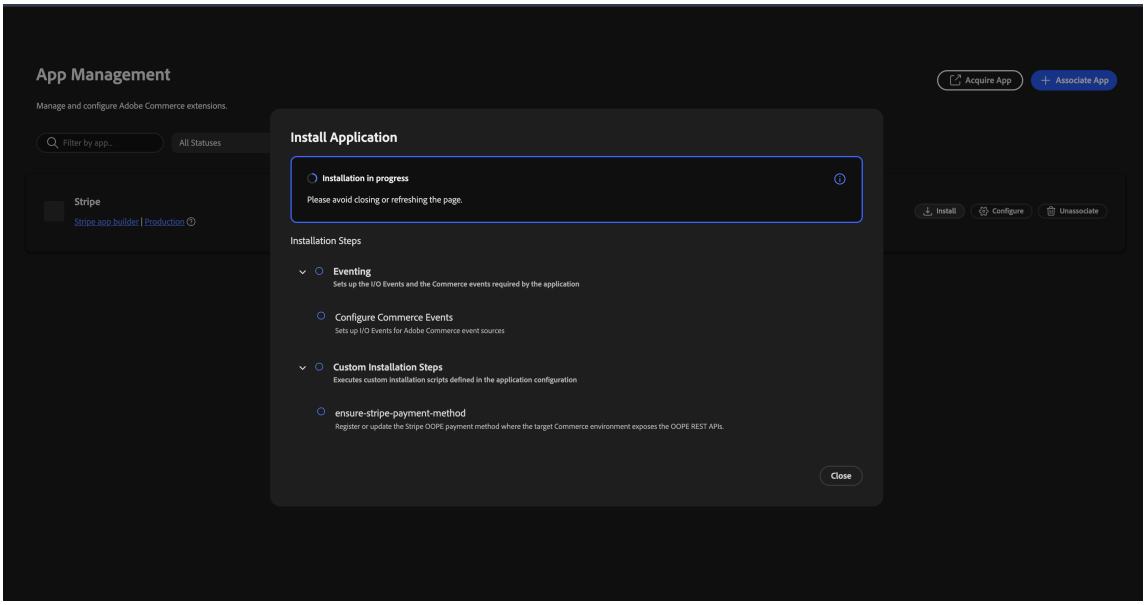
1. Log into your Adobe Commerce admin area.
2. Navigate to **System** > **Apps** > **App Management**.
3. Click **Associate App**.



1. Select the deployed Stripe app.
2. Complete the association flow.
3. Review the installation details and click **Install**.



1. Wait for App Management to finish the installation.



For more information, see Adobe's App Management guide.

SaaS storefront action discovery

The storefront must obtain `getInitParamsUrl` and `createPaymentIntentUrl` from the Stripe payment method's Commerce GraphQL `oope_payment_method_config.backend_integration_url` value. Do not hardcode an App Builder action URL or use a temporary storefront override in production. After installation, retrieve an available payment method from Commerce and verify that its returned URLs target the deployed Stripe actions.

Configure Stripe Webhooks

Configure the Stripe webhook after deployment:

1. Open Stripe Webhooks.
2. Create a webhook endpoint for the deployed App Builder action:

   ```
   https://<runtime-namespace>.adobeioruntime.net/api/v1/web/stripe/stripe-webhook
   ```

3. Subscribe the endpoint to these events:

   ```
   charge.captured
   charge.refunded
   charge.succeeded
   charge.dispute.created
   payment_intent.canceled
   payment_intent.requires_action
   payment_intent.payment_failed
   payment_intent.succeeded
   ```

These events support delayed payment methods, manual capture updates, cancellations, disputes, and refunds.

4. Save the endpoint.
5. Copy the signing secret from this exact webhook endpoint.

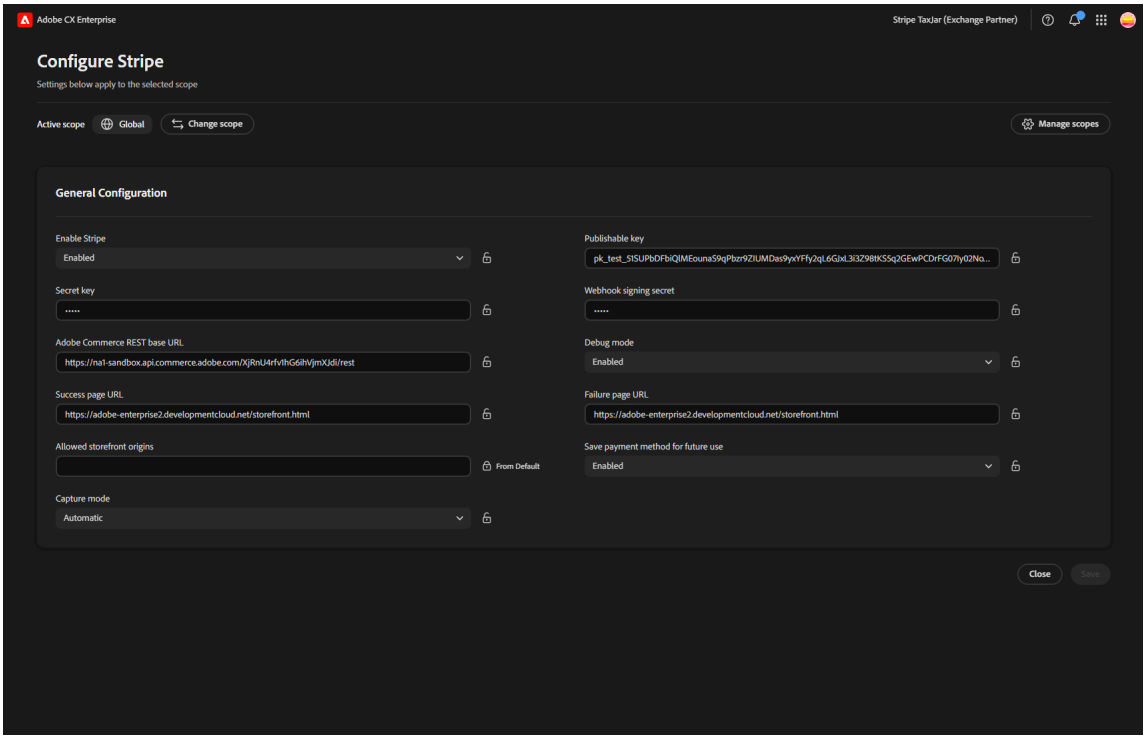
Do not reuse a webhook signing secret from another endpoint. Stripe creates a different signing secret for each webhook destination.

For more information, see Stripe's webhook endpoint guide.

Stripe Configuration

Configure Stripe from App Management:

1. In **System** > **Apps** > **App Management**, open the installed Stripe app.
2. Click **Business Configuration** or **Configure**.



The **Secret key** and **Webhook signing secret** are saved encrypted with `AIO_COMMERCE_CONFIG_ENCRYPTION_KEY`, the key you created in Install the App via Adobe Exchange Marketplace and App Management. If you change that key later, see App Management Configuration Fails to Load After an Encryption Key Change.

The following settings are available for configuration:

- **Enable Stripe**: Enables or disables the payment method at your storefront. It can be set per scope, see Enable or disable Stripe per scope.
- **Publishable key and Secret key**: Stripe account API keys used to collect payments. You can find them in Stripe API keys.

- **Webhook signing secret**: Stored with the App Management configuration for merchant reference. The deployed webhook action currently verifies Stripe signatures with the `STRIPE_WEBHOOK_SECRET` environment value, so set that deployment value to the signing secret from the exact webhook destination.
- **Adobe Commerce REST base URL**: The REST endpoint for your Commerce instance. For SaaS instances, find it in Adobe Experience Cloud Commerce instances. For PaaS instances, use your Commerce REST base URL.
- **Debug mode**: Returns additional runtime error details while testing.
- **Success Page URL**: URL where customers are redirected after successful payment when using redirect-based payment methods (iDEAL, Bancontact, PayPal, etc.). When webhook order placement completes before the browser return, the app appends `orderRef` and `orderNumber`. If order placement is still pending, the app appends `payment_intent` and `payment_status` so the storefront can continue showing a pending or confirmation state.
- **Failure Page URL**: URL where customers are redirected if payment fails when using redirect-based payment methods. Supports `{error_message}` placeholder. Example: `https://your-store.com/checkout/failure?error={error_message}`
- **Allowed storefront origins**: Comma-separated list of exact HTTPS storefront origins that may call the checkout endpoints. Leave it empty to use the `ALLOWED_ORIGINS` deployment value. The origins of the Success Page URL and Failure Page URL are always allowed.
- **Save payment method for future use**: Allows customers to save supported payment methods.
- **Capture mode**: Choose automatic capture or manual capture. Automatic capture captures payment during checkout. Manual capture authorizes payment during checkout and captures it when the Commerce order is invoiced.

Enable or disable Stripe per scope

To hide Stripe for one website or store view, open the installed Stripe app in App Management, select that scope with **Change scope**, set **Enable Stripe** to **Disabled**, and save.

App Management installs a Commerce webhook with the app. Each time Commerce lists the payment methods of a cart, the webhook asks the app whether to hide Stripe. The app reads **Enable Stripe** for the store view of the cart and removes the Stripe payment method from the list when it is set to **Disabled**. No further step is needed. The app caches App Management configuration for up to 5 minutes, so a change can take up to 5 minutes to reach checkout.

To apply a change at once, clear the cached configuration. Run the commands from the app root directory that contains `app.config.yaml` and `.env`. The cache is kept separately in each Runtime region, so clear every region:

```
aio app state delete --match "configuration.*" --region amer
aio app state delete --match "configuration.*" --region emea
aio app state delete --match "configuration.*" --region apac
aio app state delete --match "configuration.*" --region aus
```

This removes only the cached copies. The saved configuration is kept, and the next checkout reads it again.

- Only a value that was saved as **Disabled**, on the store view or on a scope above it, hides Stripe. A scope that never saved the value keeps Stripe available.
- The store view must exist in the App Management scope tree. If it was created after the scopes were last synced, use **Manage Scopes** to sync them. A store view that is not in the

scope tree is never filtered.
- If the webhook fails or does not answer in time, the list is left unchanged and Stripe stays available.
- An installation made before this webhook existed must run the App Management installation once more from **System** > **Apps** > **App Management**, so that App Management installs the webhook. Confirm that the webhook `stripe_stripe_payment_method_filter` is listed in Commerce Admin afterwards.

Checkout endpoint CORS is restricted to configured storefront origins. By default, the app derives allowed origins from the configured HTTPS Success Page and Failure Page URLs. Set **Allowed storefront origins** in the Business Configuration, or the `ALLOWED_ORIGINS` deployment value, to a comma-separated list of exact HTTPS storefront origins when the checkout runs on another domain. A value saved in **Allowed storefront origins** replaces the `ALLOWED_ORIGINS` value instead of adding to it, so list every origin there. A browser request from an unlisted origin is intentionally denied; an allowlisted origin receives the underlying application error response for troubleshooting.

Redirect Payment Methods

The Stripe integration supports redirect-based payment methods in addition to standard card payments. These include:

- **iDEAL** (Netherlands)
- **Bancontact** (Belgium)
- **SEPA Direct Debit** (Europe)
- **Giropay** (Germany)
- **EPS** (Austria)
- And many more...

To enable these payment methods:

1. Log in to your Stripe Dashboard
2. Navigate to **Settings** > **Payment methods**
3. Enable the payment methods you want to accept
4. Configure the Success and Failure Page URLs in App Management.

How redirect payments work:

When a customer selects a redirect payment method:

1. Cart is configured with shipping and billing information
2. Payment Intent is created with the payment method
3. Customer is redirected to the external payment provider to authorize payment
4. After authorization, customer is redirected back to your store
5. The `payment_intent.requires_action` webhook records the redirect context
6. After authorization, `charge.succeeded` places the Commerce order from the recorded context
7. The customer is redirected to your Success or Failure Page URL with `orderRef` and `orderNumber` when available

The Stripe webhook endpoint must subscribe to `payment_intent.requires_action` for redirect payment methods. Without this event, the app cannot record the redirect context, and later `charge.succeeded` events without order metadata are acknowledged but do not place the Commerce order.

> ***Notice:*** *If a redirect payment succeeds in Stripe but no Commerce order is created, open the Stripe webhook delivery for the* `charge.succeeded` *event. A* `202` *response with a message such as* `No order reference in Payment Intent metadata` *usually means the app did not receive the earlier* `payment_intent.requires_action` *event or could not load the current Stripe configuration. Confirm the webhook endpoint subscribes to all required events, verify that* `STRIPE_WEBHOOK_SECRET` *and the App Management Stripe API keys belong to the same Stripe account and mode, and clear the App Management configuration cache with* `aio app state delete configuration.global` *only if a saved configuration value remains stale.*

Saved payment methods are scoped to the customer and supported payment method type.

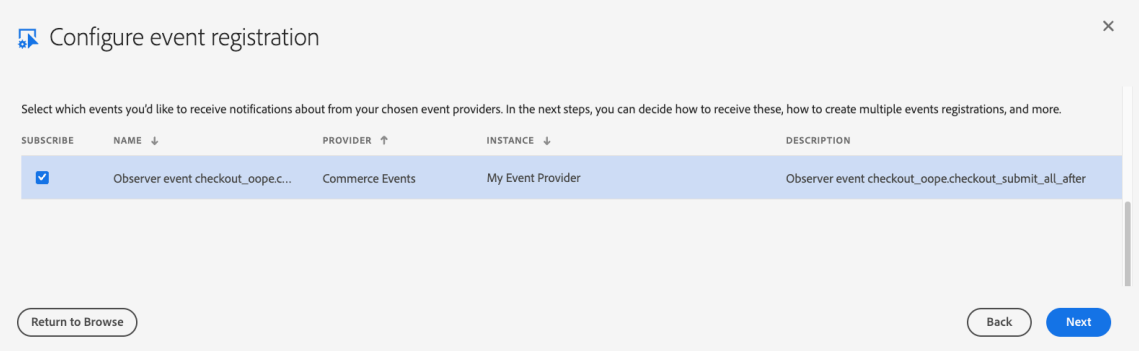
Events Configuration

Commerce events allow the app to update Stripe payments when orders, invoices, and credit memos change in Commerce. With App Management, these events are created automatically when the app is installed. The event definitions come from the app's Commerce configuration and are delivered to the private `stripe/events-handler` Runtime Action; do not create a public webhook registration for these events.

During installation, App Management shows the Commerce Events step before the custom installation step. On both SaaS and PaaS, that custom step registers or updates the Stripe OOPE payment method (`oope_stripe`) through the Commerce OOPE REST API, with `backend_integration_url` pointing at the deployed `init-params` and `payment-intent` actions.

On PaaS, the installation additionally removes and re-adds the three Stripe Commerce event subscriptions listed below so that they are bound to the Stripe events provider. This step is skipped on SaaS.

On the **Configure event registration** page, select the generated event provider, click **Next**, select the Commerce events to subscribe to, and click **Next** again.



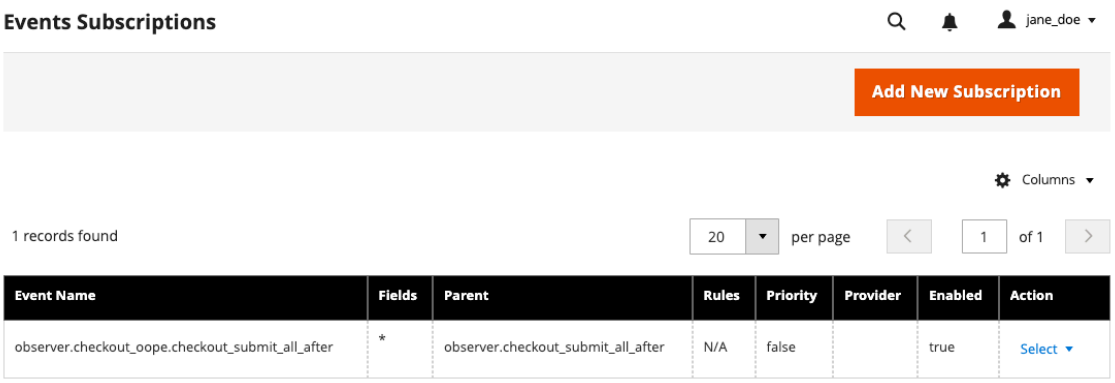
For reference, the app installs these Commerce events:

```
observer.sales_order_save_after
observer.sales_order_invoice_save_after
observer.sales_order_creditmemo_save_after
```

After installation, verify the generated event registration from Commerce Admin:

1. Go to **System** > **Apps** > **App Management**.

2. Open the installed Stripe app.
3. Click **Events**.
4. Confirm the event provider is present and the subscribed events include the order, invoice, and credit memo observers listed above.



Use the **Events** button only to review the generated event registration. For more information, including PaaS manual setup when App Management is not managing the registration, see Adobe's Adobe I/O Events for Adobe Commerce documentation.

Validate the installation

Confirm that the encryption key works: open the installed Stripe app in **System** > **Apps** > **App Management**, click **Business Configuration**, save the Stripe settings, and confirm that the page reloads without a **Configuration load error**.

Run a test checkout before enabling live payments:

1. Open the Commerce storefront.
2. Create a cart and add a product.
3. Add customer email, shipping address, billing address, and shipping method.
4. Select Stripe as the payment method.
5. Confirm the payment with a Stripe test payment method.
6. Place the order.
7. Confirm the order is created in Commerce.
8. Confirm the payment appears in the Stripe payments dashboard.
9. Confirm the order identifiers appear on the Stripe payment metadata or description.

Validate the admin flows that apply to the store:

- **Automatic capture**: Place an order and confirm the payment is captured during checkout.
- **Manual capture**: Set capture mode to manual, place an order, then create an invoice in Commerce Admin to capture the payment.
- **Stripe Dashboard capture**: If a manual-capture payment is captured directly in Stripe, confirm Commerce records an order comment. This does not create a Commerce invoice.
- **Refunds**: Create a full or partial credit memo in Commerce Admin and confirm the refund appears in Stripe. See Stripe refunds.
- **Saved payment methods**: Enable saved payment methods, place a test order, and confirm the supported payment method is saved in Stripe.

Add Stripe to your EDS storefront

You can add the Stripe payment method at the checkout page of your EDS storefront by utilizing a pre-built EDS block, which includes all functionality needed to render the payment form, collect the payment and place the order.

You can find this EDS block at https://github.com/stripe/stripe-magento2-releases/tree/master/app-builder/blocks/stripe-payment. Instructions on how to integrate it with your EDS storefront can be found inside the README.md file at the same location.

Troubleshooting

App Management Configuration Cache

If App Management saves successfully but checkout or Commerce events still use older Stripe configuration, clear only the App Management global configuration cache key for the selected App Builder workspace:

Run the command from the app root directory that contains `app.config.yaml` and `.env`:

```
aio app state delete configuration.global
```

Do not run `aio app deploy --force-events` for this issue.

Unable to Read App Management Configuration

Stripe runtime actions fail with an error that starts with `Unable to read App Management config:` when they cannot read the saved configuration. Match the rest of the message:

- `No saved App Management configuration found`: no Stripe configuration has been saved. Open the installed Stripe app in App Management, enter the Stripe settings, and save. The app does not read configuration saved on the removed legacy Stripe Configuration page.
- `Encryption key has not been given`: `AIO_COMMERCE_CONFIG_ENCRYPTION_KEY` is not set in the deployed app. Run `npx aio-commerce-lib-config encryption setup` from the Stripe app project root directory, then deploy again.
- `The given encryption key is not a valid hex string.` or `The given encryption key must be 64 hex characters (32 bytes).`: the key in `.env` is malformed. Run `npx aio-commerce-lib-config encryption validate`, correct the value, and deploy again. If secrets were already saved, restore the original key instead of generating a new one.
- `Failed to decrypt value`: the deployed key differs from the key that encrypted the saved secrets. See the next section.

App Management Configuration Fails to Load After an Encryption Key Change

If the Stripe configuration page shows **Configuration load error**, and the browser also displays a FormatJS message such as `An id must be provided to format a message`, inspect the `app-management/config` runtime activation first. An error similar to the following confirms that the saved password fields cannot be decrypted with the currently deployed `AIO_COMMERCE_CONFIG_ENCRYPTION_KEY`:

```
Failed to decrypt value
Unsupported state or unable to authenticate data
```

The FormatJS message is a secondary App Management UI error and is not the root cause.

If the previous encryption key is available, restore that key to the same App Builder workspace and redeploy after receiving deployment approval. This preserves the saved secrets. If the previous key is unavailable, clear only the encrypted Stripe fields with the configuration action's `PATCH` endpoint, then enter the secrets again in App Management.

1. In the browser developer tools, open the failing `GET /api/v1/web/app-management/config` request and record its runtime host and `scopeId`. Use the scope UUID from the request; do not use `global` as the scope ID.

2. Obtain a current Adobe IMS access token and organization ID for the same organization and workspace.

3. Set the values locally without committing them or placing the token directly in shell history:

```
export APP_RUNTIME_HOST="your-runtime-host.adobeioruntime.net"
export APP_SCOPE_ID="00000000-0000-0000-0000-000000000000"
export ADOBE_IMS_ORG_ID="YOUR_ORG_ID@AdobeOrg"
read -rsp "Adobe IMS access token: " ADOBE_IMS_TOKEN
export ADOBE_IMS_TOKEN
echo
```

4. Unset only `privateKey` and `webhookSecret`:

```
curl --fail-with-body --request PATCH \
  "https://${APP_RUNTIME_HOST}/api/v1/web/app-management/config" \
  --header "Authorization: Bearer ${ADOBE_IMS_TOKEN}" \
  --header "x-api-key: exc_app" \
  --header "x-gw-ims-org-id: ${ADOBE_IMS_ORG_ID}" \
  --header "Content-Type: application/json" \
  --data "{\"scopeId\":\"${APP_SCOPE_ID}\",\"config\":
[{\"name\":\"privateKey\",\"value\":null},
{\"name\":\"webhookSecret\",\"value\":null}]}"
```

   A successful request returns HTTP `200` with `Configuration values updated successfully`. Other configuration fields remain unchanged. The removed secret values cannot be recovered without the original encryption key or another secure copy.

5. Verify that the same request used by App Management loads successfully:

```
curl --fail-with-body \
  "https://${APP_RUNTIME_HOST}/api/v1/web/app-management/config?
scopeId=${APP_SCOPE_ID}&commerceEnv=saas" \
  --header "Authorization: Bearer ${ADOBE_IMS_TOKEN}" \
  --header "x-api-key: exc_app" \
  --header "x-gw-ims-org-id: ${ADOBE_IMS_ORG_ID}"
```

6. Refresh App Management, re-enter the Stripe secret key and webhook signing secret for the affected scope, and save. Clear the local token when finished:

```
unset ADOBE_IMS_TOKEN
```

Never paste an IMS access token into documentation, source control, tickets, or chat. Revoke or expire any token that is accidentally exposed.

[PaaS Only] Installation Fails with 404 Error

If the App Management installation fails with `Failed to register Stripe OOPE payment method` and the Commerce message `Request does not match any route.`, confirm that `magento/module-out-of-process-payment-methods` is installed and enabled.

[PaaS Only] Authorization Error during Installation

If the App Management installation fails with `The consumer isn't authorized to access Magento_OutOfProcessPaymentMethods::oope_payment_method_create`, update the Commerce integration API permissions for out-of-process payment methods.

[PaaS Only] Module Installation Issues

If PaaS module installation or `setup:upgrade` fails, verify module versions, module enablement, and file permissions from the Adobe Commerce application root directory:

```
# Run from the Adobe Commerce application root directory, where bin/magento exists
composer show magento/module-out-of-process-payment-methods
composer show magento/commerce-eventing
composer show magento/commerce-backend-sdk
composer show magento/adobe-ims

php bin/magento module:status Magento_OutOfProcessPaymentMethods
php bin/magento module:status Magento_AdobeCommerceEventsClient
php bin/magento module:status Magento_AdobeCommerceEventsGenerator
php bin/magento module:status Magento_AdobeCommerceEventsMeta
php bin/magento module:status Magento_AdminUISDK
php bin/magento module:status Magento_AdobeIms
php bin/magento module:status Magento_AdminAdobeIms
```

If a required module is disabled, enable it and apply Commerce setup changes:

```
# Run from the Adobe Commerce application root directory, where bin/magento exists
php bin/magento module:enable \
   Magento_OutOfProcessPaymentMethods \
   Magento_AdobeCommerceEventsClient \
   Magento_AdobeCommerceEventsGenerator \
   Magento_AdobeCommerceEventsMeta \
   Magento_AdminUISDK \
   Magento_AdobeIms \
   Magento_AdminAdobeIms

php bin/magento setup:upgrade
```

```
php bin/magento setup:di:compile
php bin/magento cache:flush
```

If generated classes, cache, or static content cannot be written, fix ownership and permissions for the Commerce writable directories. Replace `www-data` with the web server user used by the Commerce instance, such as `magento`, `apache`, or `nginx`:

```
# Run from the Adobe Commerce application root directory, where bin/magento exists
sudo chown -R www-data:www-data var generated pub/static pub/media
find var generated pub/static pub/media -type d -exec chmod 775 {} \;
find var generated pub/static pub/media -type f -exec chmod 664 {} \;
php bin/magento cache:flush
```

Authentication Issues

If you cannot access the Stripe configuration, confirm that the app is installed in Adobe Commerce App Management and the selected Adobe Developer Console workspace matches the deployed app.

Webhook and Event Issues

If Stripe webhooks do not process, confirm the endpoint is active in Stripe Webhooks, the deployed `STRIPE_WEBHOOK_SECRET` belongs to that exact endpoint, and the endpoint receives `2xx` responses.

If redirect payments succeed in Stripe but do not create Commerce orders, check these items first:

1. Confirm the Stripe webhook endpoint is subscribed to `payment_intent.requires_action` and `charge.succeeded`.
2. Confirm the deployed `STRIPE_WEBHOOK_SECRET` belongs to the same webhook endpoint that receives the events.
3. Confirm the publishable and secret keys are from the same Stripe account and mode as the webhook endpoint.
4. After changing App Management configuration, clear the runtime configuration cache from the app root directory:

   ```
   aio app state delete configuration.global
   ```

5. Retry the redirect payment with a new cart.

If Commerce events do not process, verify the App Management installation and generated event registration:

1. In Commerce Admin, go to **System** > **Apps** > **App Management**.
2. Open the installed Stripe app and confirm the app status is installed.
3. Click **Events**.
4. Confirm the generated provider is present.
5. Confirm the subscribed events include `observer.sales_order_save_after`, `observer.sales_order_invoice_save_after`, and `observer.sales_order_creditmemo_save_after`.

6. In Adobe Developer Console, open the same project and workspace, then review the event registration connected to the app runtime action.

For PaaS manual setup when App Management is not managing the registration, use Adobe's Adobe I/O Events for Adobe Commerce documentation.

[PaaS Only] Commerce Events Not Firing

If Commerce events do not reach the app on PaaS after order, invoice, or credit memo activity, run these commands from the Adobe Commerce application root directory, where `bin/magento` exists.

Confirm the subscribed events:

```
bin/magento events:list -vv
bin/magento events:registrations:list -vv
```

The Stripe app expects these three namespaced events on the Commerce Stripe events provider:

```
stripe.observer.sales_order_save_after
stripe.observer.sales_order_invoice_save_after
stripe.observer.sales_order_creditmemo_save_after
```

If any are missing, remove and re-add them. Set `STRIPE_PROVIDER_ID` to the Stripe provider ID shown in Commerce Admin under the installed Stripe app **Events** page, or from `bin/magento events:registrations:list -vv`:

```
bin/magento events:unsubscribe stripe.observer.sales_order_save_after
bin/magento events:unsubscribe stripe.observer.sales_order_invoice_save_after
bin/magento events:unsubscribe stripe.observer.sales_order_creditmemo_save_after

bin/magento events:subscribe stripe.observer.sales_order_save_after \
  --parent observer.sales_order_save_after \
  --providerId "${STRIPE_PROVIDER_ID}" \
  --fields=entity_id --fields=increment_id --fields=quote_id \
  --fields=customer_firstname --fields=customer_lastname --fields=customer_email \
  --fields=grand_total --fields=status --fields=state \
  --fields=payment.method --fields=payment.additional_information \
  --fields=payment.additional_data --force

bin/magento events:subscribe stripe.observer.sales_order_invoice_save_after \
  --parent observer.sales_order_invoice_save_after \
  --providerId "${STRIPE_PROVIDER_ID}" \
  --fields=entity_id --fields=order_id --fields=increment_id \
  --fields=grand_total --fields=state \
  --fields=order.entity_id --fields=order.increment_id --fields=order.quote_id \
  --fields=order.payment.method --fields=order.payment.additional_information \
  --force

bin/magento events:subscribe stripe.observer.sales_order_creditmemo_save_after \
  --parent observer.sales_order_creditmemo_save_after \
  --providerId "${STRIPE_PROVIDER_ID}" \
```

```
  --fields=entity_id --fields=order_id --fields=invoice_id \
  --fields=increment_id --fields=grand_total --fields=state \
  --fields=order.entity_id --fields=order.increment_id --fields=order.quote_id \
  --fields=order.payment.method --fields=order.payment.additional_information \
  --force
```

Synchronize event metadata and clear cache:

```
bin/magento events:metadata:populate
bin/magento cache:flush
```

You can also click **Execute Synchronization** on **Stores** > **Settings** > **Configuration** > **Adobe Services** > **Adobe I/O Events** > **General configuration**.

Confirm Adobe I/O Events is enabled in that Admin page, and that cron is running. While testing, you can publish queued events immediately:

```
bin/magento cron:run
# or
bin/magento queue:consumers:start commerce.eventing.event.publish
```

On Adobe Commerce on cloud infrastructure, also confirm `.magento.env.yaml` enables eventing:

```yaml
stage:
  global:
    ENABLE_EVENTING: true
  deploy:
    CRON_CONSUMERS_RUNNER:
      cron_run: true
      max_messages: 0
      consumers: []
```

Place or update a new order after these steps. Existing carts or already-processed documents may not emit another useful event.

Feature Comparison

Feature	SaaS	PaaS
IMS Authentication	Required	Recommended
Integration Auth	Not Available	Alternative
Pre-installed Modules	Yes	Manual Install
Admin UI SDK	Pre-configured	Requires Setup
EDS Storefront	Direct Support	Additional Config
Luma Storefront	Not Available	Available

For more information about PaaS compatibility, see Adobe's Extension Compatibility Guide.

Release Notes

Version 1.3.0

Version 1.3.0 lets merchants enable or disable Stripe per website or store view from App Management.

Enable Stripe per scope

- **Enable Stripe** now applies per scope. Setting it to **Disabled** for a website or store view removes Stripe from the payment methods Commerce returns for carts of that store view. Other store views keep Stripe.
- App Management installs a Commerce webhook (`payment_method_filter.get_list`) with the app. Commerce calls it each time it lists the payment methods of a cart, and the app answers from the saved **Enable Stripe** value of the cart's store view.
- Only a value saved as **Disabled** hides Stripe. If the setting cannot be read, or the webhook fails or times out, Stripe stays available.
- A change can take up to 5 minutes to reach checkout because of the configuration cache. See Enable or disable Stripe per scope to apply it at once.
- Existing installations must run the App Management installation once more so that the webhook is installed.
- The app version is aligned at 1.3.0 across the App Management configuration, the extension manifest, the package metadata, and the Stripe application info reported to Stripe.

Configuration through App Management only

- The legacy **Stripe Configuration** page in the Commerce Admin menu and its separate configuration storage were removed. Stripe is configured only in App Management, and values saved on the legacy page are no longer read.
- **Allowed storefront origins** is now a Business Configuration field. It replaces the setting that was only available on the legacy page.
- `AIO_COMMERCE_CONFIG_ENCRYPTION_KEY` is required, because the app has no other configuration source. Create it with `npx aio-commerce-lib-config encryption setup` and keep the same value for the workspace. See Install the App via Adobe Exchange Marketplace and App Management.
- `ENCRYPTION_KEY` , `LEGACY_ENCRYPTION_IV` and the `npm run generate-encryption-key` script are no longer used. You can remove the two values from `.env` .
- The archived legacy setup scripts and guidance are no longer part of the package.

Version 1.2.1

Version 1.2.1 is a maintenance and reliability release for version 1.1.0. It corrects the Stripe payment method registration on Adobe Commerce as a Cloud Service, makes refund and PaymentIntent handling idempotent, repairs Commerce event bindings on PaaS, and aligns the supported Node.js versions and dependencies.

Payment method registration

- The App Management installation step now registers or updates the Stripe out-of-process payment method (`oope_stripe`) on both SaaS and PaaS. Previously this step was skipped on SaaS, so the payment method could be missing from the available payment methods returned for a cart.
- The `backend_integration_url` returned by Commerce GraphQL (`oope_payment_method_config.backend_integration_url`) points at the deployed `init-params` and `payment-intent` actions.

- Existing SaaS installations that do not return `oope_stripe` should update or reinstall the app from **System** > **Apps** > **App Management** so that the installation step runs again.

Refunds

- Before creating a refund, the app checks Stripe for an existing refund tagged with the Commerce credit memo ID and skips creation when one already exists.
- Refund creation is skipped when the payment is already fully refunded in Stripe, for example when the refund was initiated from the Stripe Dashboard.
- Every refund request carries an idempotency key derived from the credit memo ID, which prevents duplicate refunds when Stripe or Commerce deliver the same event more than once.
- Credit memo events that do not include an `entity_id` are rejected with a `400` response.

Order and PaymentIntent handling

- Repeated Commerce order-save events (for example after an invoice or credit memo) no longer update a PaymentIntent that already has the order metadata attached.
- The PaymentIntent for an order is resolved from the `client_secret` stored on the order payment first. The quote lookup is used only when the PaymentIntent was never confirmed or the `client_secret` is unavailable. This prevents mismatches when quote or order IDs are reused across Commerce databases.
- PaymentIntents in the `succeeded`, `requires_capture`, and `processing` statuses are treated as confirmed.

Express Checkout

- The capture method is now passed as a top-level Stripe Elements option instead of a per-payment-method option. This resolves Express Checkout initialization failures when the capture mode is set to manual.

PaaS Commerce events

- The installation adds an `ensure-stripe-commerce-events` step that removes and re-adds the order, invoice, and credit memo event subscriptions so that they are bound to the Stripe events provider. The step is skipped on SaaS.
- A **[PaaS Only]** troubleshooting section, "Commerce Events Not Firing", documents how to verify and re-create the subscriptions with `bin/magento`.

Compatibility and maintenance

- The supported Node.js versions are `>=22.23.4 <23.0.0` and `>=24.22.1 <25.0.0`.
- Dependencies were updated to address `npm audit` findings.
- Legacy manual setup scripts for events and payment methods were retired from the npm scripts and archived as historical reference. Installation and configuration are performed through App Management.
- The app version is aligned at 1.2.1 across the App Management configuration, the extension manifest, the package metadata, and the Stripe application info reported to Stripe.
- A `deploy` npm script was added (`aio app deploy --force-build --force-deploy`).

Version 1.1.0

Version 1.1.0 introduced installation and configuration through Adobe Commerce App Management, including the Business Configuration form, generated Commerce event subscriptions, and the App Management installation flow for SaaS and PaaS.